

Bratis and McEvilly (2019) investigated the content and structure needed to specify resilience requirements. Resilience requirements often take the form of a resilience scenario. There can be many such scenario threads in the ConOps or OpsCon. The following information is often part of a resilience scenario:

- Operational concept/scenario name
- System or system element of interest
- Capability(s) of interest their metric(s) and units
- Target value(s) (required amount) of the capability(s)
- System modes of operation during the scenario (e.g., operational, training, exercise, maintenance, update)
- System states expected during the scenario
- Adversity(s) being considered, their source, and type
- Potential stresses on the system, their metrics, units, and values (Note: Stresses are a type of adversity. They are proximate forces or influences, directly affecting the system that can cause degradation of the system's ability to deliver required capability.)
- Resilience related scenario constraints (e.g., cost, schedule, policies, regulations)
- Timeframe and sub-timeframes of interest
- Resilience metric(s), units, determination method(s), and resilience metric target(s) (e.g., expected availability of required capability, maximum allowed degradation, maximum length of degradation, total delivered capability). Note: There may be multiple resilience targets (e.g., threshold, objective, As Resilient as Practicable (ARAP)).

Importantly, many of these parameters may vary over the timeframe of the scenario. [Figure 3.7](#) notionally shows the required capability, the stress on the system, and the delivered capability as they vary as a function of time. A single resilience scenario may involve multiple stresses, which may be involved at multiple times throughout the scenario.

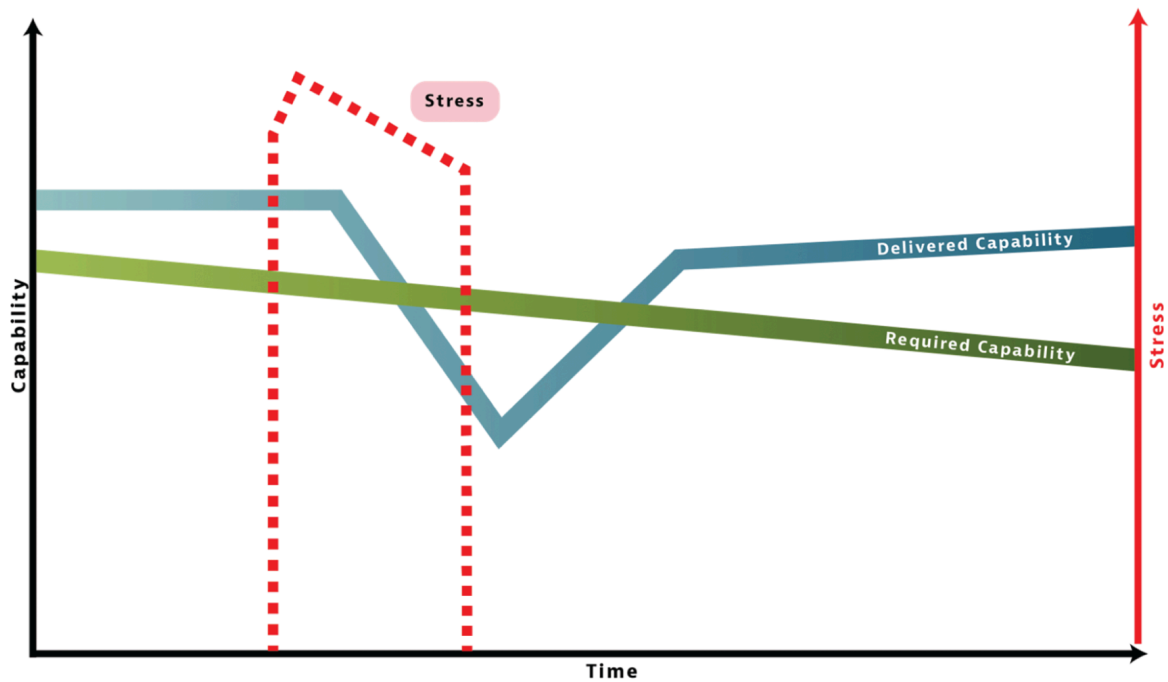


FIGURE 3.7 Timewise values of notional resilience scenario parameters. INCOSE SEH original figure created by Brtis and Cureton. Usage per the INCOSE Notices page. All other rights reserved.

Requirement Patterns for Resilience

An example of a natural language pattern for representing this information would be:

The<system, mode(t), state(t)>encountering<adversity(t), source

Here, “(t)” is used to indicate that the item may vary as a function of time.

System State Considerations

When a system encounters an adversity, the system may pass through a number of states, from a fully capable state to a state minimally acceptable to the system stakeholders. Intermediate states include damaged or partially capable states. The transitions between these states fall into three categories. The *robustness* category defines the transitions for which the system maintains its level of capability. These transitions include maintaining fully capable or partially capable states. The *tolerance* category

includes passing from any higher level of capability to a lower level of capability (e.g., a degraded capability). The *recovery* category includes passing from any lower-level capability to a higher-level capability including the original fully capable state. The system should be designed applying design principles that will manage the transitions between states to result in context appropriate behavior. Guidance on techniques is provided in Jackson and Ferris (2013) and Brtis (2016).

Related Quality Characteristics

Resilience has commonality and synergy with a number of other QCs. Examples include availability, maintainability, reliability, safety, security, and sustainability. This group of quality areas are referred to as loss-driven areas because they all focus on potential losses involved in the development and use of systems (see [Section 3.1.13](#)). These areas frequently share: the assets considered, losses considered, adversities considered, requirements, and architectural, design, and process techniques. It is imperative that these areas work closely with one another and share information and decision-making in order to achieve a holistic approach that avoids unbalanced emphasis in any one area.

Further information and references on resilience (including the state-of-the-art) can be found in the resilience section of the SEBoK.

3.1.10 Sustainability Engineering

Definition

Sustainability Engineering is an approach that supports the circular economy over its life.

Key Concepts

Design for sustainability is defined as the process that considers environmental and social aspects as key elements in product design to

reduce the harmful impacts of the product throughout its life cycle (Sharma, et al., 2020). It entails environmentally conscious decisions that promote responsible disposability via product recycling and materials reuse as options for the preservation of scarce material resources. Sustainability and disposability are critical components toward the circular economy, which is based on a production and consumption model that involves sharing, reusing, repairing, and recycling existing products and materials as much as possible, expanding the life cycle of products, minimizing waste and pollution, and creating a closed-loop system (Geissdoerfer, et al., 2020). These goals are consistent with the 17 Sustainable Development Goals that were adopted by all UN Member States in 2015, as part of the 2030 Agenda for Sustainable Development (Haskins, 2021).

Elaboration

Role of Sustainability and Disposability in SE

Addressing sustainability effectively in context is highly complex, requiring the integration of multiple disciplines in balancing a wide range of interdependent issues (Pearce, et al., 2012). Sustainability is essential given the significant impacts that the SE processes, and the resulting systems, have had, and continue to have, on the environment. Achieving sustainability must include a holistic adoption of environmental stewardship in engineering activities (Alwi, et al., 2014) (Rosen, 2012).

The focus of environmental impact analysis is on potential harmful effects of a proposed system's development, production, utilization, support, and retirement stages. Concern extends over the full life cycle of the system, from the materials used and scrap waste from the production process, operation of the system, replacement parts, consumables and their packaging, to final disposal of the system.

Disposal analysis is a significant analysis area within environmental impact analysis. During System Architecture Definition and Design Definition (see

Sections 2.3.5.4 and 2.3.5.5), one goal is to maximize the economic value of the residual system elements after useful life and minimize the amount of waste materials. Design for disassembly has become an important consideration in the design process so that the products are created in a way that minimizes destructive separation of system elements such that the material can be reused in future generations of products, remanufacturing, or recycling processes (Abuzied, et al., 2020). This may include designing for transformation (e.g., decomposition, biodegradation).

Key Activities, Tools, and Methods of Sustainability and Disposability

The ISO 14000 (2015) series of environmental management standards are an excellent resource for methods to analyze and assess industrial operations and their impacts on the environment. Attention to environmental regulations should be addressed in the earliest activities of requirements analysis. The Øresund Bridge (see [Section 6.2.](#)) is an example of how early analysis of potential environmental impacts ensures that measures are taken in concept, development, and production to protect the environment with positive results. Two key elements of the success of this initiative were the continual monitoring of the environmental status and the integration of environmental concerns into the requirements of the two countries.

Another effort in the ISO community is the development of a standard for Environmental Product Declarations (EPD), based on carbon footprints, as an indicator of the global environmental impact of a product expressed in carbon emission equivalents (He, et al., 2018). EPD and labeling, such as the Nordic Swan and Blue Angel, offer consumers assistance in their purchasing decisions. Methods associated with life cycle assessment (LCA), life cycle impact assessment (LCIA), life cycle optimization (LCO), and life cycle management (LCM) are increasingly sophisticated and supported by software (Avraamidou, et al., 2020).

Related QCs

Achieving a circular closed-loop system relies on integrating additional quality characteristics. Useful life extensions rely on reliability and maintainability (see [Section 3.1.8](#)) alongside efficient logistical support (see [Section 3.1.6](#)) and products designed to be resilient (see [Section 3.1.9](#)). Recovery of valuable resources after useful life is highly dependent on decisions made when considering manufacturability (see [Section 3.1.7](#)).

More information on Sustainability/Disposability can be found in the Journal of Cleaner Production (2023), the Journal of Environmental Management (2023), Wood et al. (2023), ICE (2023), and MDPI (2023).

3.1.11 System Safety Engineering

Definition

System safety engineering is an approach that reduces the likelihood of harm to people, assets, and the wider environment.

Key Concepts

The goal of system safety engineering is to reduce and mitigate hazards of systems to an acceptable level of risk. Engineered systems have safety risks; they are not 100% safe. The definition of what is acceptably safe, safety regulations, processes, and culture vary across different industries and countries.

System safety engineering is not limited to ensuring that the engineered system is acceptably safe. It includes minimizing the risks to everyone involved in the production, utilization, support, and retirement of the system, as well as third parties who could also be affected by these activities. System safety engineering is about engineering the SoI, the wider socio-technical operational system, and the socio-technical (or even purely social) management system.

Safety is an emergent property of the engineered system in its real operational environment. How the system is used, maintained, and

managed can have as big an impact on system safety as its inherent design. Understanding, and aligning, the mental models of designers, operators, and managers is critical.

Safety is managed by minimizing the hazards that can lead to an accident. This is either through reducing the likelihood the hazard will occur or minimizing the impact if it does. This is either through designing the hazard out, technical mitigations in the system, or procedural controls. This requires a mixture of suitably qualified people, effective processes, appropriate governance, and culture.

In-service systems need careful monitoring to ensure that design assumptions remain valid, no new hazards have been identified, and that operations/maintenance is as expected. Slow feedback loops and misaligned mental models can be particularly problematic, as issues can grow unseen for years before they appear with catastrophic impact.

Good systems safety engineering seeks to ensure operators do not misuse systems, leaders set the right tone and culture, and maintainers don't take shortcuts. System safety engineers and senior leaders are often accountable for predictable misuse of systems, failure to address poor behavior, and ineffective oversight of maintenance and operations.

Elaboration

Acceptably Safe

The safety regulations, processes, and culture vary across different industries and countries. What is acceptable in one industry and country may not be acceptable in another. There is a wide diversity in regulators, definitions, evidence, and perceived benefits that adds further complexity. Even the definition of what is “acceptably safe” varies. Typical perceptions of “acceptably safe” include:

- “*We have complied with the necessary [product] regulations.*” This is generally accepted for simple, well understood, standardized system

elements (e.g., electrical cable, bolts).

- “*We have evaluated all identified hazards and have mitigated each to be ‘as low as reasonably practicable’ (ALARP).*” This would be the typical approach adopted for complicated, safety critical civilian systems (e.g., a railway signaling system, passenger aircraft).
- “*We have evaluated all the identified hazards of the system, and they are either ALARP, or the level of hazard of the new system is less than the alternative (of not having it).*” This would be the typical approach adopted for military, medical, or emergency response systems (e.g., artillery, pacemaker).

Eliminating all safety risk is not possible; therefore, no system can be described as 100% safe. There may be unknown hazards and hazards that cannot be eliminated but are determined to be acceptable given the perceived benefit. Similarly, assuming that the system must be safe because there haven’t been any accidents yet, or reported, is equally incorrect. There may have been near misses (see below), the hazardous functionality/performance may not have been used, or the effect/damage has not yet been recognized.

Emergence, Accidents, and Hazards

Safety is an emergent property of a system. It is not the sum of system element level safety or reliability. Rather, it is impacted by interactions between system elements and affected by the environment in which it is used. Many factors affect the level of safety risk (e.g., who uses and maintains a system, how they do it, in what environment). Safety is not a static property as systems (and the surrounding ecosystems) are dynamic and evolve. The designers’ expectations of risks often differ from the system under test and evaluation and from the system in operation (which continues evolves over time). The mental models of the humans interacting with the system and the related processes are also dynamic and subject to change.

Most accidents result from more than a single causal factor. When an unmitigated hazardous situation does occur but does not result in harm, it is referred to as a *near miss*. Near misses serve as critical feedback on the system safety level in operation. Safety culture often determines how near misses are treated and responded to. However:

- Accident investigations often reveal dependencies between hazards exist, despite earlier beliefs that causal factors were independent. A failure to recognize dependencies when applying statistical methods can result in flawed safety decisions.
- A hazardous situation can occur without a system element failure. This could be because of a design error, an implementation error, or a misalignment between mental models of designer and operators or maintainers.

Regulators typically assess safety risk in terms of both:

- the *likelihood* of hazards occurring and leading to harm and
- the *severity* of the resulting harm.

Safety is managed by eliminating hazards where possible; and when not possible, by reducing risks to an acceptable level. When possible, design changes to eliminate potential hazards are the preferred options. The next preferred options are design mitigations to reduce the likelihood of hazards occurring. When designs changes or mitigations are not possible, other means are typically employed such as operational controls and limitations, maintenance inspections or activities, warnings via labeling, and training.

Hazards may result from a range of sources such as intrinsic, functional, socio-technical, or management/wider culture. Intrinsic hazards are typically caused by the material, or other design factors of the system elements used in the system. Functional hazards result from incorrect, unexpected, or undesirable functions or performance of the system. Socio-technical hazards result from interactions between the physical system and its operators and the wider environment. Finally, management/cultural

hazards relate to the system and the wider management controls needed to realize and sustain the system.

Examples of safety hazards include:

- Interactions between system elements, the operating environment, and operators: A car on an icy road and an inexperienced driver is likely to result in an accident. Traction control, trained drivers, or not driving in icy conditions mitigates this hazard.
- Mistakes in system/system element requirements, design, manufacturing, or installation: A failure to specify the Maneuvering Characteristics Augmentation System (MCAS) system element as safety critical resulted in loss of two 737 Max aircraft (Cantwell, 2021).
- The system creates hazards in the wider SoI: Bull-bars / kangaroo bars reduce the risk of injury to a driver in vehicle to vehicle or vehicle to large animal collisions. However, they significantly increase the risks to pedestrians in vehicle to pedestrian collisions in urban areas (Desapriya, et al., 2012).
- The inherent material used in system elements: Asbestos or flammable substances present inherent hazards.
- Incorrect operation or maintenance: A failure to properly remove old wiring led to the Clapham junction rail accident (Hidden, 1989).
- Misaligned mental models between operators, maintainers, and designers: Prior to the Smiler accident, false alarms were a common occurrence. This led to operators believing all alarms were false positives. This resulted in alarms being overridden without investigation (Kemp and O'Neil, 2018).
- The activities undertaken to design, manufacture, test and maintain the system: The Piper-Alpha oil platform accident was caused by a failure to properly manage design changes, poor maintenance management, and poor contingency planning (Cullen, 1990).

Managing and Controlling Hazards

System safety engineering seeks to control hazards by:

- Understanding the system environment, wider SoI, proposed system, and how it will be used.
- Specifying the safety requirements for the system. System requirements flow from stakeholder needs and requirements and are derived into system element requirements (see [Section 2.3.5.3](#)). Maintaining traceability is key, as it may not be immediately obvious to a system element designer that a specific requirement is safety critical (see [Section 3.2.3](#)).
- Analyzing the potential safety hazards. There is a range of hazard analysis techniques looking at the system, and its functions, physical, process, and human interactions. An effective hazard analysis will use multiple techniques.
- Mitigating/controlling the known hazards, either by reducing the likelihood or severity of a hazard occurring. Approaches include removing the hazard entirely, designing-in passive or active controls to mitigate the hazard, or including operational or maintenance controls. A key element of including controls is ensuring effective feedback loops and recognizing the full control model beyond the engineered system (including the operating environment and management systems) (Leveson, 2011).
- Establishing a safety management system to ensure the system remains safe throughout its life cycle.

Establish an Appropriate Safety Management System

Each organization's safety management system needs to be tailored based on country, region, and industry/application considerations. Organizations need to understand the regulatory, operational, and physical environments that the systems they develop will be used. For example, the safety management system needed for a safety critical industry such as rail or aerospace would be inappropriate for consumer electronics.

The safety management system needs to: define the organizations approach to developing safe systems; manage the infrastructure, processes and information required to support system delivery; oversee the operations

and maintenance of in-service systems; and ensure an effective safety culture.

The safety management system needs to be fully integrated into the wider organizations' business management systems. It needs to ensure that:

- Projects to deliver new systems are given clear safety objectives, measures, and targets.
- In-service systems are operated and maintained safely, with appropriate monitoring of changes to use, environment, and material state of the system.
- Incidents and near misses are reported without fear of retribution, and on-going monitoring and implementation of mitigation actions create an ongoing learning/improvement cycle (Dekker, 2014).

A key facet of the safety management system is the organization's safety and ethical culture, or "how we behave when no-one is looking" (see [Section 5.1.4](#)). Regular measurement of the safety is useful to track organization's safety culture (Hudson, 2001). Regular, frequent communications and stories of the behavior needed are necessary by senior and influential people to reinforce the safety culture (Kemp and O'Neil, 2018).

Establish and Run Projects with Safety at Their Core

Key issues to be aware of include:

- Ensuring that the tailoring of the SE approach is appropriate for the system under development.
- Ensuring that the safety engineering processes integrated as tightly as possible to the wider engineering and business processes, driving both operational efficiencies and reducing the risk of incorrect assumptions between the safety team and the wider project.
- Accepting the need to make assumptions, but recognizing that when the assumptions turn out to be incorrect and appropriate rework will be

required (and accepting the cost and time impact of the rework).

- Ensuring the selection of an appropriate life cycle model. As Akroyd-Wallis (2018) notes, direct adoption of Agile software techniques to safety critical systems may be problematic. SE Practitioners should exercise caution when employing agile for safety critical systems.
- Terminating projects when they can no longer meet their safety objectives at a reasonable cost, timescales, or performance.

Embed Safety in the Core of the SE Process

Specific safety engineering considerations include:

- Ensuring Business or Mission Analysis (see [Section 2.3.5.1](#)) includes a high-level assessment of hazards and that analysis of alternatives includes their inherent safety potential
- Including a comprehensive hazard analysis during Stakeholder Needs and Requirements Definition (see [Section 2.3.5.2](#)) to identify potential hazards with the system being developed and the wider operational capability the SoI will be deployed into
- Ensuring that key safety functions and safety performance is captured during System Architecture Definition (see [Section 2.3.5.4](#)). A safety viewpoint will help ensure traceability from high level needs down to system element requirements.
- Where the system is to be part of a wider SoS, ensuring that system functions/performance necessary to mitigate SoS hazards are captured as safety requirements (see [Section 4.3.6](#)).
- Ensuring that verification and validation of safety requirements is sufficiently rigorous to meet the agreed levels of “acceptably safe.”
- Ensuring that hazards that are managed by Operational or Maintenance (see Sections 2.3.5.12 and 2.3.5.13) activities are clearly embedded in relevant processes and are clearly communicated, precisely defined, and reasonable.
- Ensuring that the assumptions in the safety case remain valid through life. (Is the environment as expected? Have the operators (and their mental models) changed? Has the use of the system changed? Is

maintenance being done as required and is the maintenance as effective as planned? Are there any new potential new hazards being seen? Is the frequency and severity of hazards as expected?)

Ensure the System Is Delivered Safely

System safety engineering needs to ensure that systems can be designed, built, and verified safely. The systems that manufacture, test, and maintain complicated and complex systems can be as hazardous as the SoI itself. The traditional split between product safety and occupational safety is becoming less clear as:

- Development becomes more agile.
- Organizations shift from product to service delivery.
- Increased use of non-expert operators and maintainers.

Suitably Qualified and Experienced Personnel Drive Safety Performance

The effectiveness of safety management system is driven by the people who work within it. Good people may build safe systems despite poor processes and tools. Good processes and tools cannot make up for poorly qualified people.

Effective safety practitioners need to be numerate, critical thinkers, and system thinkers who understand the technologies being used, the environment the system will be deployed into, and the mental models of operators and maintainers. In addition, they need the influencing and persuasion skills to convince others of the right approach to take and the moral courage to “say no” when necessary (see [Section 5.1.2](#)).

Safety practitioners need to be in the room when key decisions are made. They need to lead the safety decision making, capturing key information in an audit trail. If they are involved too late, organizations can be left with systems that cannot be deployed. This happened to the UK Air Force when acquiring eight Chinook Mk3 helicopters (NAO, 2008).

For more illustrations on the importance of system safety, refer to Case 1 (Radiation Therapy - The Therac-25) from [Section 6.1](#) and Case 5 (Artificial Intelligence in Systems Engineering – Autonomous Vehicles) from [Section 6.5](#).

3.1.12 System Security Engineering

Definition

System Security Engineering is an approach that identifies, protects from, detects, responds to, and recovers from anomalous and disruptive events, including those in a cyber contested environment.

Key Concepts

System Security Engineering (SSE) is focused on ensuring a system can function under anomalous and disruptive events associated with misuse and malicious behavior. SSE involves a disciplined application of SE principles in analyzing security threats and vulnerabilities to the system and assessing and mitigating security risks to assets of the system during the life cycle. It blends technology, management principles and practices, and operational rules to ensure sufficient protections are available at all times.

Sources of potential anomalous and disruptive events (threats) are many and varied. They may emanate from external sources (e.g., theft, denial of service attacks, power interruptions) or may be caused by internal forces (e.g., user actions, supporting systems). A disruption may be unintentional (misuse) or intentional (malicious) in nature

Physical security protects a system from unauthorized access, misuse, or damage caused by physical actions and events such as theft, vandalism, and intrusion. Protecting physical facilities, equipment, resources, and personnel can involve the use of multiple layers of interdependent systems such as surveillance and intrusion detection systems, deterrent systems, security guards, protective barriers, locks, and access control. Hardware

devices can employ anti-tampering features to detect unauthorized opening or altering of the packaging, either to ensure the content is authentic or to trigger actions to protect sensitive information in the devices.

As our world becomes increasingly digital, both hardware and software systems are increasingly at risk for disruption or damage caused by threats taking advantage of digital technologies. Integrating and implementing systems security using SSE approaches is the most efficient and effective way to ensure that security is addressed at each stage of the life cycle and becomes part of the overall SE solution instead of being done separately and isolated from other SE activities (NIST SP 800-160 Vol. 1, 2022 and NIST 800-160 Vol. 2, 2021). SSE provides the needed complementary engineering capability that extends the notion of trustworthiness to deliver trustworthy secure systems, which are less susceptible to the effects of modern adversity such as attacks orchestrated by an intelligent adversary (NIST SP 800-160 Vol. 1, 2022).

Cybersecurity generally refers to the confidentiality, integrity, and availability of information assets. Security management includes controls (e.g., policies, practices, procedures, organization structures, and software). Trustworthiness is a concept that includes privacy, reliability, resilience, safety, and security, therefore worthy of being trusted to fulfill whatever critical requirements may be needed for a particular system element, system, network, application, mission, business function, enterprise, or other entity (NIST 800-160 Vol. 2, 2021).

Elaboration

SSE practitioners should have skills, expertise, and experience in multiple areas. Examples include security requirements, security architecture views, threat assessment, networking, security technologies, hardware and software security, security test and evaluation, vulnerability assessment, penetration testing, and supply chain security risk assessment. A major challenge in managing engineering projects is unclear security roles, responsibilities, and accountability. To assist in the security role

development and understanding responsibilities, an SE/SSE roles and responsibilities framework can be used to break down tasks into a matrix format that enables the SE practitioner to understand the role contributions and identify the types of artifacts created by the execution of the SE life cycle processes. (Nejib, et al., 2017)

Through NIST 800–160 Vol. 1 (2022) and Vol. 2 (2021), it has been determined that the best way to integrate cybersecurity into systems is through an SE process. NIST 800–160 Vol. 1 (2022) and Vol. 2 (2021) are based on ISO/IEC/IEEE 15288 (2023) and this handbook. They use the same terminology so that both SE and SSE practitioners can understand the key relationship that exists between the two disciplines. There is a direct correlation between SE and SSE, and SE practitioners need to understand and incorporate security components into each SE life cycle process. [Table 3.4](#) shows an example of how the SSE technical processes in NIST SP 800–160 Vol. 1 (2022) can be reused and referenced by SE, SSE, and other disciplines practitioners. Specifically, [Table 3.4](#) is an example of the Implementation process (see [Section 2.3.5.7](#)) breakout defined with extensions for SSE to include the purpose, outcomes, activities and tasks, inputs, and responsible and supporting roles. This same format was used to breakout each of the technical SSE processes in NIST 800–160 Vol. 1 Rev. 1 (2022) and the SE Technical Processes (see [Section 2.3.5](#)) to build an understanding of the relationships between the SE and SSE processes.

TABLE 3.4 Implementation process breakout

Implementation Process Breakout

Purpose

- Realize the security aspects of the system element
- Results in a system element that satisfies specified system security requirements, architecture, and design

Outcomes

- Security aspects of the implementation strategy are developed
- Security aspects of implementation that constrain the requirements, architecture, or design are identified
- Security system element
- System elements securely packaged and stored
- Enabling systems or services needed for security aspects of implementation
- Traceability of security aspects of implemented system elements

Activities and Tasks

1. Prepare for the security aspects of implementation
2. Perform the security aspects of implementation
3. Manage results of the security aspects of implementation

Inputs	Security strategy, plan, traceability, requirements, design, architecture, secure system elements, assurance evidence, assurance results, and anomalies report
---------------	--

Responsible and Supporting Roles	Responsible: Systems Security Engineer (SSE) Supporting: Program Manager (PM), Chief Engineer (CE), Systems Engineer (SE), Systems Architect (SA), and Test Engineer (TE)
---	--

From NIST 800–160 Vol. 1 (2022). Used with permission. All other rights reserved.

Case Study 3, Cybersecurity Considerations in Systems Engineering-The Stuxnet Attack on a Cyber Physical System (see [Section 6.3](#)) provides an example of the importance of system security.

3.1.13 Loss-Driven Systems Engineering

Loss-Driven Systems Engineering (LDSE) is the value adding unification of the QCs that address the potential losses associated with developing and using systems (Brtis, 2020). SE methodologies often focus on the delivery of desired capability. As a result, SE methodologies are largely capability-driven, and may not provide integrated attention to the potential losses associated with developing and using systems. Loss and loss-driven QCs are often considered in isolation—if at all. Examples of loss-driven QCs include resilience, safety, security, sustainability/disposability, and availability.

There is significant commonality and synergy among the loss-driven QCs, which needs to be leveraged. To do this, work on the loss-driven QCs should be collaborative on:

- the adversities considered,

- the weakness, defects, flaws, exposures, hazards, and vulnerabilities considered,
- the assets and losses considered, and
- the coping mechanism considered.

Further, SE practitioners should:

- elicit, analyze, and capture loss-driven requirements as an integrated part of the overall stakeholder and system requirements development,
- make architectural and design decisions holistically across the loss-driven QC areas, and
- integrate the management of risks associated with all loss-driven areas into the project's risk management activities.

3.2 SYSTEMS ENGINEERING ANALYSES AND METHODS

Part II of this handbook provided a set of SE life cycle processes used across the system life cycle. Each process contains a set of process activities and elaborations in the context of that specific life cycle process. This section provides insight into topics, techniques, and methods that cut across the SE life cycle processes, reflecting various aspects of the concurrent, iterative, and recursive nature of SE.

3.2.1 Modeling, Analysis, and Simulation

Overview and Purpose

The INCOSE Systems Engineering Vision 2035 (2022) predicts that “The future of systems engineering is model-based, leveraging next generation modeling, simulation, and visualization environments powered by the global digital transformation, to specify, analyze, design, and verify systems. High fidelity models, advanced visualization, and highly integrated, multidisciplinary simulations will allow SE Practitioners to

evaluate and assess an order of magnitude more alternative designs more quickly and thoroughly than can be done on a single design today.”

The essential artifact of modeling, analysis, and simulation (MA&S) is an explicit model, an idealized representation of one or more aspects of the as-is or to-be SoI. Systems Modeling and Simulation has been defined (NAFEMS and INCOSE, 2019) as the use of interdisciplinary functional, architectural, and behavioral models (with physical, mathematical, and logical representations) for all life stages.

The terms “modeling,” “analysis,” and “simulation” are sometimes used interchangeably. However, they clearly refer to distinct activities. *Modeling* is the conception, creation, and refinement of models. *Analysis* is the process of systematic, reproducible examination to gain insight. *Simulation* is the process of using a model to predict and study the behavior or performance of the SoI—for aspects represented in the model. Simulation is often performed to support a particular kind of analysis, but not all analysis is performed through simulation. In the classification of models presented below, two major types of models are distinguished: *physical models*, and *digital models*. The SE discipline primarily makes use of digital models, since they provide many benefits to the SE processes in a timely and affordable manner, in particular during the early life cycle stages. Other engineering disciplines make use of both physical and digital models throughout the product life cycle. Although sometimes the term “simulation” is used in conjunction with a *physical model*, in this section *simulation* always involves a *digital model*, and any examination involving a *physical model* is always a *test*.

For effective MA&S, digital models are often parameterized to enable analysis or simulation of multiple configurations or situations with one model. Each configuration is typically defined by grouping selected parameter values in an *experiment*, see also Minsky (1965). Alternatively, the terms “analysis case,” “load case,” or “verification case” are used, depending on the application domain or convention. An *experiment* specifies the purpose of the analysis or simulation as well as the

combination of target scenario, environment, initial and boundary conditions, and any other user-defined parameters. [Figure 3.8](#) shows a typical workflow.

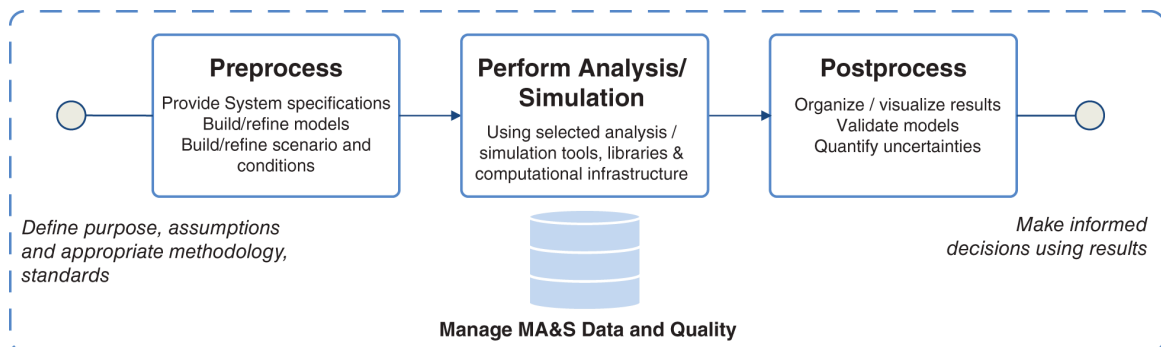


FIGURE 3.8 Schematic view of a generic MA&S process. INCOSE SEh original figure created by the NAFEMS-INCOSE Systems Modeling and Simulation Working Group (SMSWG)). Usage per the INCOSE Notices page. All other rights reserved.

MA&S links directly to the Core Competency “Systems Modeling and Analysis” in the INCOSE SE Competency Framework (INCOSE SECF, 2018). MA&S supports all Technical, Management and Integrating Competencies. Appropriate MA&S practices can clearly support SE professionals (individuals and teams) to perform better SE, more efficiently.

MA&S Related to Life Cycle Processes

Creation and refinement of descriptive models in the Business or Mission Analysis process (see [Section 2.3.5.1](#)) and the Stakeholder Needs and Requirements Definition process (see [Section 2.3.5.2](#)) can be used to ensure that the business or mission proposition is understood correctly, the problem is understood correctly, is specified at the appropriate level of detail, and is fully shared with the stakeholders. MA&S can be used in the System Requirements Definition process (see [Section 2.3.5.3](#)) to flow-down the system requirements to system elements. This may include models that specify functional, interface, performance, and physical requirements, as well as other nonfunctional requirements (e.g., reliability, maintainability, safety, and security). In addition to bounding the system design parameters, MA&S can also be used to validate that the system requirements reflect stakeholder needs and requirements before proceeding with subsequent life cycle processes.

MA&S can be used in the System Architecture Definition process (see [Section 2.3.5.4](#)) concurrently with the Design Definition process (see [Section 2.3.5.5](#)) to synthesize and define alternative system concepts, compare and evaluate candidate options, and enable discovery of the best architecture and design, including the integration with other systems and unambiguously defining the system's capabilities and the value it is expected to deliver to its stakeholders (e.g., in the form of MOEs and MOPs). MA&S is often used extensively to realize an iterative model-based or model-driven design workflow. In many application domains, it is much more cost—and schedule-effective to perform analysis and simulation with digital models than to prototype with physical models. Digital models also allow for full and continuous access to all model parameters and properties, which is often infeasible with physical models. MA&S lends itself to fast iterations between problem specification, architectural design, detailed design, and V&V, as well as between system elements at different levels of decomposition. Using MA&S in the System Analysis process ([Section 2.3.5.6](#)), related system analyses can be used to explore a trade space by modeling alternative system solutions, or even generate many candidate solutions, and assessing the impact of critical properties such as mass, speed, energy consumption, accuracy, reliability, and cost on the overall adequacy and performance.

MA&S can be used in the Implementation process (see [Section 2.3.5.7](#)) to support definition, understanding and prediction of behavior for various aspects of the enabling production (manufacturing) and supply chain processes for the envisaged SoI. Models that reflect the “as produced” state of the SoI can be used to develop production facilities (factory) and “digital twins.” MA&S can be used in the Integration process (see [Section 2.3.5.8](#)) to support integration of the elements into a system, as well as in the Verification process (see [Section 2.3.5.9](#)) to support verification that the system satisfies its requirements. This often involves integrating lower-level hardware and software design models with system-level design models, thereby allowing verification that the system requirements are satisfied. Systems integration and verification may also include replacing selected hardware and design models with actual hardware and software

products to incrementally verify that system requirements are satisfied: so-called hardware-in-the-loop and software-in-the-loop testing. In cases where testing is impossible or carries prohibitive cost, verification of the SoI can be done by analysis or simulation using high fidelity digital models. Models can be used to simulate relevant operational environments where actual environments are unattainable, too costly, or not reproducible. Simulation can use observed data as inputs for computation of critical parameters that are not directly observable.

In the Operation process (see [Section 2.3.5.12](#)), MA&S can support definition, understanding, and prediction of behavior for various aspects of the envisaged or actual operation of the SoI, to help train (future) users to interact with the system, and to develop training material. Models may form a basis for developing a simulator of the system with varying degrees of fidelity to represent user interaction in different usage scenarios. Models and simulators can also be used to perform dry runs to prepare for complex or risky operations with the real, deployed system. In the Maintenance process (see [Section 2.3.5.13](#)), models that reflect the “as maintained” state of deployed systems can be used to develop “digital twins,” possibly for individual deployed systems. Such models can be connected to data acquisition in the field and provide valuable insight and support for health monitoring and preventive maintenance. They can also be used to plan system upgrades and evolutions. MA&S in the Disposal process (see [Section 2.3.5.14](#)) can be used to predict and monitor system disposal. MA&S also enables model-based iterations between the development processes mentioned above, as depicted in [Figure 3.9](#).

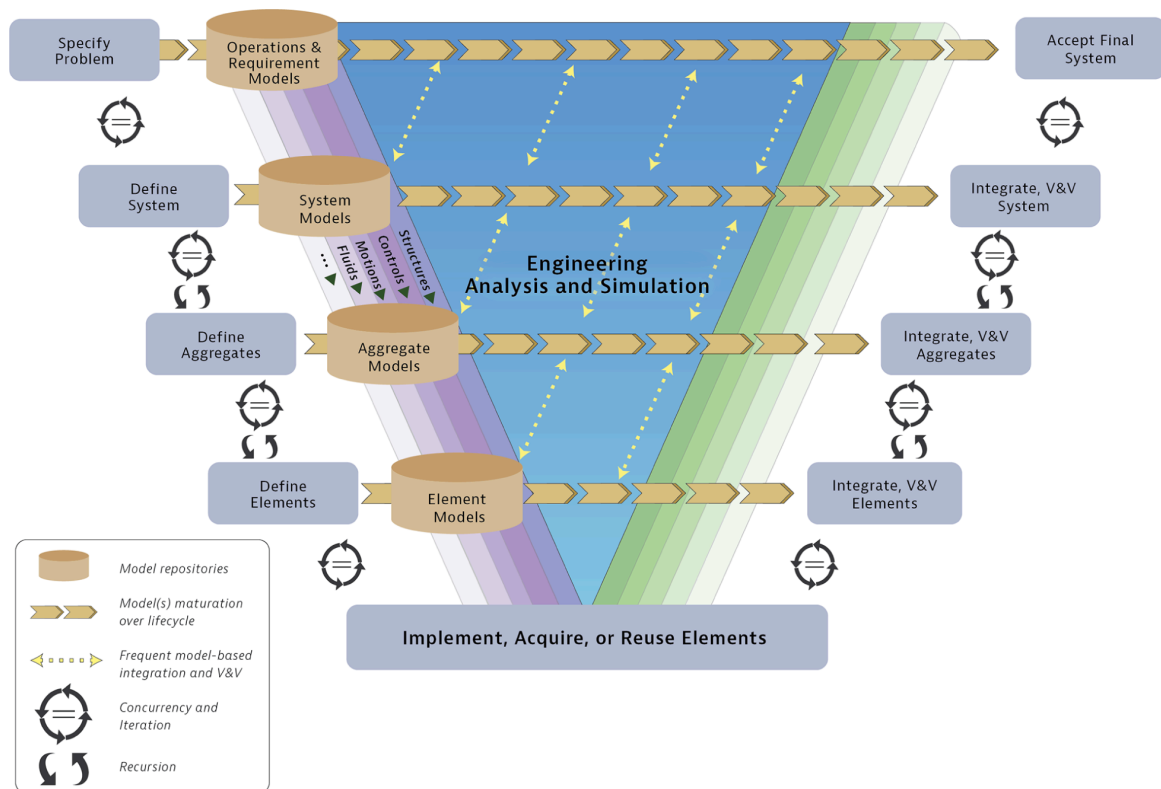


FIGURE 3.9 System development with early, iterative V&V and integration, via modeling, analysis, and simulation. Derived from NAFEMS and INCOSE (2019), based on NDIA, et al. (2011). Used with permission. All other rights reserved.

Cross-Cutting MA&S

There are several cross-cutting uses of MA&S that are not tied to a particular life cycle process, including:

Characterizing an existing system—Existing systems may be poorly documented (in whole or in part). Modeling such a system can provide a concise way to capture its architecture and design. This model can then be used to facilitate maintenance of the system or its further evolution.

Knowledge transfer within teams—MA&S enables the creation and maintenance of more precise, elaborate, and consistent specifications, including the rationale behind many requirements or design choices. Capturing specifications in rich models helps to mitigate the risk of loss of knowledge in case of team changes in long-duration projects.

Automated mapping and transformation—Digital models can be transformed by declarative or procedural algorithms (i.e., automated

generation of a new or modified digital model from an existing one). Model transformations are a very powerful means to increase the value of model-based engineering (e.g., convert model formulations from one modeling language to another, move from a systems architecture to a (partial) software architecture, package a model for use outside the owning organization by encapsulating and protecting its intellectual property, creation of a surrogate model from a much more detailed discipline-specific engineering model). The value of transformation increases even more when it can be made bi-directional.

Knowledge capture and system design evolution—MA&S can be an effective means for capturing knowledge about a system as part of the Knowledge Management process (see [Section 2.3.3.6](#)). Established modeling can help transform tacit into explicit organizational knowledge. MA&S in projects enables identification and capture of reusable patterns or modules from problem and solution models that have proven their worth. Catalogs of such reusable model patterns and modules can become important assets for organizational knowledge management.

Benefits

MA&S have many advantages including:

Separation of representation and presentation—Models capture representations of an SoI. MA&S tooling can then be used to maintain an “authoritative source of truth” and produce many different presentations, or views, of the model(s) that are *correct-by-construction* and enable effective communication with all engineering and non-engineering actors (humans and machines). The model-based approach can thus overcome the main problem of the document-centric approach in which representation and presentation are often combined in single information containers, which leads to a lot of information duplication and therefore cumbersome maintenance and potential errors. This is one of the most important advantages of the model-centric over the traditional document-centric approach.

More explicit SE—SE Practitioners can use MA&S to systematically check their own thinking, assumptions, and decision making with quantitative analyses. They can capture rationales and decisions in an accessible, traceable, consistent way.

Better problem specification—The stakeholder needs and requirements can be refined and formalized as an integral and structured set of goals, assumptions, requirements, constraints, actors, typical usage scenarios, and critical capabilities, with full traceability between all model elements. Anticipated system behaviors and performances can be explored and vetted with the stakeholders before proceeding with the development of an actual solution and committing significant resources.

Rigorous, well-documented design—MA&S facilitates development of solutions in a more rigorous and consistent way which leads to higher quality specifications. Systematic design space exploration and structured trade-studies become possible. Interfaces can be defined rigorously. Simulation with digital models enables design experimentation and optimization that is impossible, not affordable, or not on time with physical models. Technical and business decision-making can also be integrated into the model repositories for future consultation.

Early V&V to reduce risk—Early validation and verification of solutions with respect to the problem specification can be performed. This enables stakeholders to be informed of the implications of their preferences, provides perspective for evaluating alternatives, and builds confidence in the solution as it develops. Systematic, regular checking of interfaces and actual interconnections is feasible. It also allows catching issues early in the life cycle, when mitigation is affordable and change of scope is still feasible. The ability to detect limitations and incompatibilities early in a project helps avoid cost and schedule overruns in later life cycle stages.

Multi-user collaboration—Use of modern MA&S tooling allows for multi-user and multi-discipline collaboration with integrated configuration and version control, including splitting work into distinct parallel branches and

merging results back into a main branch, using well-established workflows. Once deployed, this workflow is much more sophisticated and effective than what could be achieved with a document-centric approach.

Better change impact assessment—Modeled specifications with traceability (see [Section 3.2.3](#)) and MA&S tooling allow for change impact assessments by highlighting the consequences of a considered change.

Improved mastering of complexity—The value of MA&S increases with the complexity of the SoI, be it functional or physical. MA&S is a means to master greater complexity by structuring, refining, evaluating, and sharing all information within integrated project teams. On-demand multiple views with dynamic filtering to a suitable level of detail are possible. Quick views to share and capture information for effective communication with other actors, such as non-engineering disciplines and stakeholders, become more feasible and affordable.

Better team planning and handover—Development, deployment, and operational staff can more easily comprehend the design specifications, appreciate imposed limits from technology and management, and ensure an adequate degree of sustainability. Adequate, accurate, and timely MA&S helps an organization and its suppliers to plan and put in place the necessary and sufficient personnel, methods, tools, and infrastructure for system realization.

Enable efficient maintenance and traceability—Digital models (supported with appropriate tools) enable bookkeeping of SE artifacts as they evolve through the life cycle stages in a reliable, consistent, traceable, and timely manner, while also providing hyperlinked navigation.

Flexible and repeatable querying—A rigorous MA&S approach enables gaining insight into many different aspects by querying the models with pertinent questions (what-if, impact of change, etc.) and getting answers efficiently, in support of decision making. Queries can generate many

different views from the modeled information that address concerns of selected stakeholders. Query formulations can also be persisted for reuse.

Classifying and Characterizing Models

There are many different kinds of models to address different system aspects and different kinds of systems. Generally, a specific type of model focuses on some subset of the system characteristics, such as timing, process behavior, measures of performance, interfaces, and connections. It is useful to classify the types of models to assist in selecting the appropriate one. [Figure 3.10](#) shows one possible (non-exhaustive) taxonomy as an example.

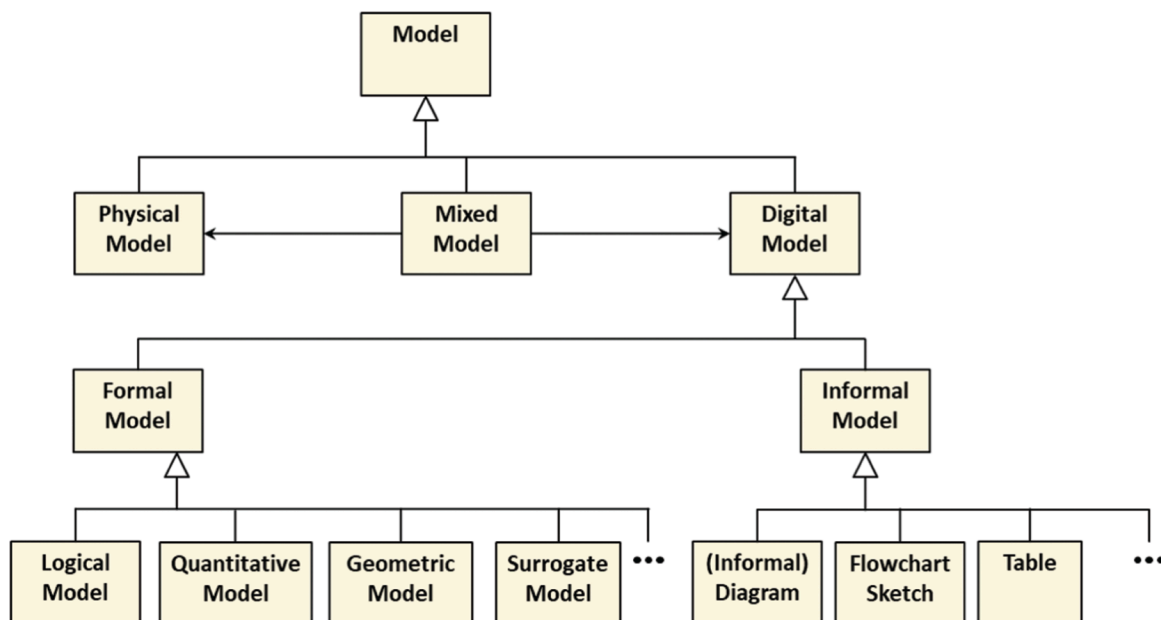


FIGURE 3.10 Illustrative model taxonomy (non-exhaustive). INCOSE SEH original figure created by the NAFEMS-INCOSE Systems Modeling and Simulation Working Group (SMSWG) derived from Friedenthal. Usage per the INCOSE Notices page. All other rights reserved.

Physical model—A physical model represents (aspects of) a system with real parts. Examples are a physical mockup, a scaled model airplane, a wind tunnel model, and a 3D-printed scale model from a digital model specification (the latter could be considered a physical view of a digital model). If simulation is performed with a physical model, it is typically called a *test*.

Digital model—Digital models can have many different expressions to represent (e.g., a system, entity, phenomenon, or process), each of which may vary in degrees of formalism. Therefore, the next level of classification is between informal and formal models.

Formal models—A formal model is expressed in a machine-readable language with explicitly defined semantics. The language may be textual and/or graphical, but with only one way of interpretation. Formal models can be further classified as logical, quantitative (i.e., mathematical), geometric, or surrogate models. A logical model, also referred to as a descriptive model or a conceptual model, represents logical relationships about the system such as whole–part relationships, interconnection relationships between elements, or precedence relationships between activities, to name a few. Logical models are often depicted using network graphs (with nodes and edges) or tables. A quantitative model represents quantitative relationships (e.g., mathematical equations) about the system or its elements that yield numerical results. A geometric model represents the geometry, geometric shapes, and spatial relationships of the system or any of its (physical) elements. A surrogate model is a reduced model that is derived from a higher fidelity, more detailed model using a data-driven, typically automated, transformation. The goal (and challenge) is to create a surrogate that adequately represents essential aspects of the modeled system while requiring substantially less computational resources. Surrogate models then enable running large numbers of (parameterized) experiments in order to facilitate design exploration, optimization, or validation.

Informal models—An informal model is expressed using some convention understood by humans, where the convention is defined casually without formal semantics. The model does not need to be machine-readable. An informal model can be created by hand or with simple tools (e.g., word-processing, spreadsheet, diagramming, mind-mapping). While such informal representations can be useful, they often lack the rigor to be considered a type of model that is truly usable for MA&S for non-trivial systems. Informal model presentations may be used as views that are

generated from or ingested into formal models in order to communicate with people not familiar with the notation.

Mixed models—A mixed model is a combination of physical and digital models.

In addition to a selected type of model, any model can be further characterized for its intended purpose through the following three characteristics:

- The *model breadth* reflects what aspects of the SoI—and possibly its (actual or intended) environment(s)—are represented, and to what extent.
- The *model granularity* characterizes the amount of visible detail captured in the model, in terms of the represented depth of system decomposition as well as the represented level of details of individual system elements.
- The *model fidelity* indicates how accurately the model represents the real-world system. Where applicable, this includes the computational precision to be achieved and the discretization scheme to be used.

The type of model and the model characteristics must be balanced against project needs and resources. Another important aspect of modeling is to explicitly state the assumptions and limitations that almost inevitably apply to any model.

Model Interoperability

Since the development of complex systems requires collaboration between all project members and disciplines, it is very important to have the ability to exchange and share models as well as analysis and simulation results across disciplines, projects, organizations, and life cycle stages. This is also referred to as *digital interchange*. In most projects and (extended) enterprises it is not possible to standardize on a single set of tools. The alternative is to develop and utilize open, tool-independent standards that

enable information exchange and sharing. There is an increasing awareness and consensus between user communities and tool developers on the merit of international royalty-free standards. Standards can be categorized in terms of how MA&S is supported. The main categories are:

- Standardized data exchange file,
- Application programming interface (API),
- Modeling language, and
- Process.

Data exchange files are used for on-demand transfer of complete models or results. APIs usually support more fine-grained data access and sharing, often implementing a service-oriented software architecture. Modeling languages can be graphical, textual or both, and are used to standardize the way of expressing a model. Process standards specify (aspects of) the MA&S processes. Most modeling languages do not prescribe a particular methodology to be followed. This flexibility is a feature of a general-purpose modeling language that enables economies of scale for implementations which are in the interest of the SE community as a whole. However, in order to align how SE practitioners in a team, organization, or application domain approach MA&S, a methodology is needed. A methodology provides guidance and examples on how to organize MA&S over a typical system life cycle, how to structure model artifacts, as well as what stages and milestones to respect. A methodology can also capture proven modeling patterns and checklists, as well as good practices in general. For further details see [Section 4.2.1](#) or consult the OMG MBSE Wiki (2023).

Tools

For physical models, the tools are generally the same as those used for production of the final SoI, plus general or dedicated test facilities. For digital models, the tools are typically MA&S software applications running on general-purpose digital computers. For some computationally intensive applications, HPC (High Performance Computing) facilities may be needed.

A MA&S software application may consist of one integrated tool or a set of tools that each implement part of the needed capabilities. The typical features of such tools are a graphical user interface with a hierarchical model structure browser, palettes of model constructs, a graphical and/or textual editor for creation and modification of the model, and multiple views for visualization, reporting, diagnostics, etc. The tool typically checks on-the-fly for adherence to the supported modeling formalism or language. If analysis or simulation support is included, there are also model execution views. For further details consult INCOSE SETDB (2021) or NAFEMS (2021).

Modeling Quality and Metrics

The quality of a model should not be confused with the quality of the design that the model represents. A perfect model can represent a bad design. On the other hand, a low-quality model can in principle represent a good design, although that is not very useful.

A completed model or simulation can be considered a system or a product in its own right. Therefore, the general steps in the development and application of a model are closely aligned to the SE processes described within this handbook. MA&S needs to be planned and tracked, just like any other developmental effort. An essential good practice is to define clearly the purpose and intended life cycle of any (type of) model upfront. In particular, verification and validation of the MA&S methods, procedures, and infra-structure themselves are essential to ensure that the resulting models, analyses, and simulations possess the required quality and credibility that make them “fit for purpose” in an application domain or project. The required rigor of the approach depends on the criticality of the SoI. As an example, the US DoD Modeling and Simulation Enterprise has developed comprehensive guidance on Verification, Validation and Accreditation (VV&A, 2021).

A valuable feature of digital models is that they are amenable to many other kinds of computation than pure analysis or simulation. This enables

the assertion of many modeling metrics such as:

- compliance with design or certification rules, including naming conventions, and associated model quality requirements,
- structural consistency of the system architecture,
- compliance of system element interconnections with interface specifications,
- coverage, consistency, and completeness of traceability, such as requirements satisfaction and verification, as well as function allocation,
- consistency and completeness of logical to physical architecture mapping and allocation,
- statistics that assist in monitoring and establishing specification maturity, uncertainty quantification, and resource planning,

MA&S Industrial Practice

A big driver for the adoption of MA&S via all engineering disciplines is the trend that complex systems are becoming more and more software intensive. Analysis and simulation using digital models is much more economical and scalable than prototyping and testing with physical models, especially in the earlier stages of the life cycle. In the later stages, a mixed approach is often used. An example of the latter is an incremental hardware-in-the-loop approach in performing dynamic simulations. When it can be justified, verification through a purely digital model may be used.

Since a major responsibility of SE is to regard the system as a whole and coordinate between all disciplines in a multidisciplinary team, it follows that SE MA&S must interoperate at some level with the modeling and simulation of each of the other engineering disciplines in a team. As shown in [Figure 3.11](#), the trend is to use an integrated system model to ensure information consistency between all engineering disciplines through a hub-and-spokes pattern, where a system model repository forms the hub.

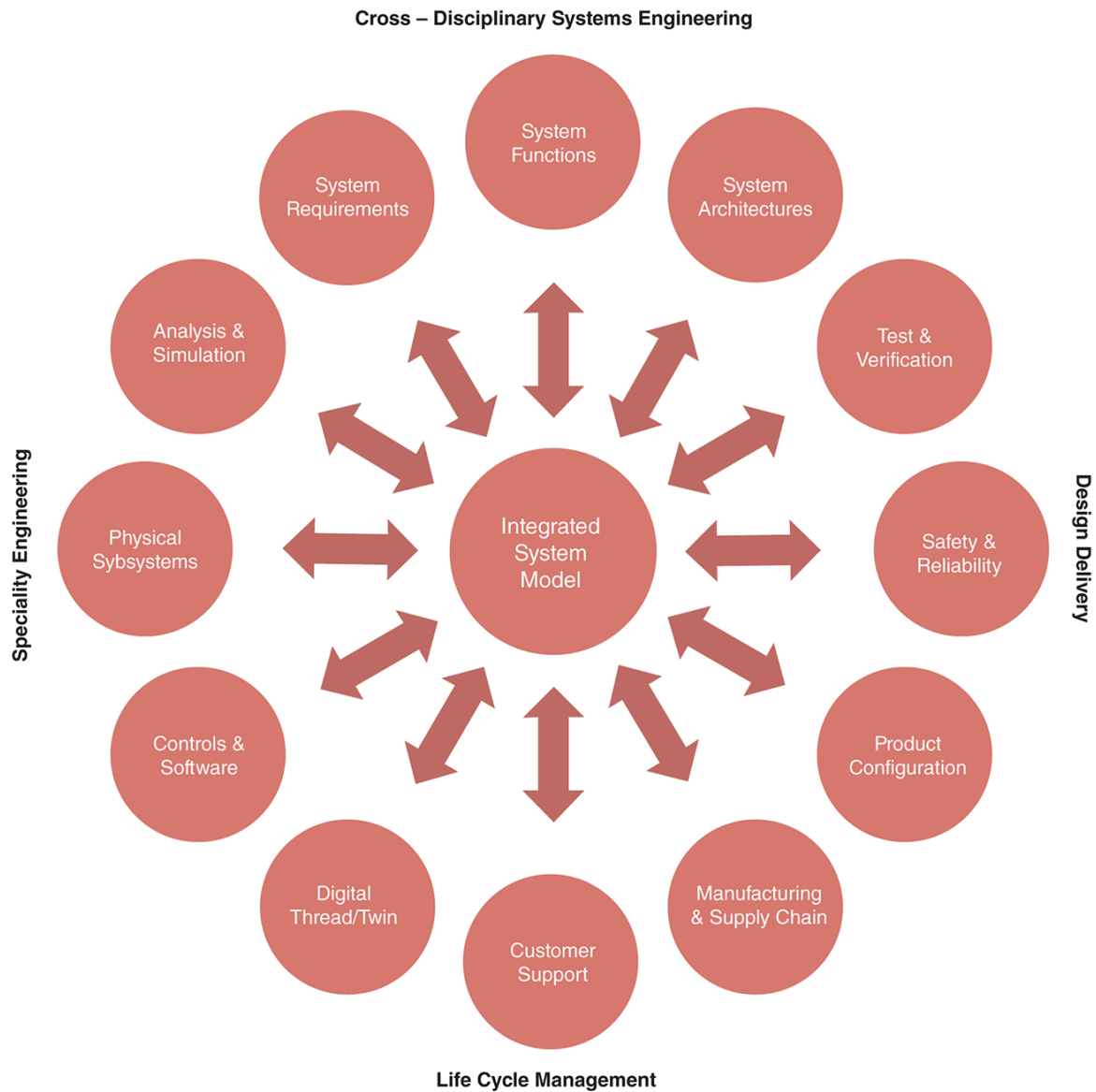


FIGURE 3.11 Model-based integration across multiple disciplines using a hub-and-spokes pattern. Derived from NAFEMS and INCOSE (2019). Used with permission. All other rights reserved.

A different way to look at multidisciplinary MA&S coordination is shown in the life cycle view presented in [Figure 3.12](#). The information needed by two or more disciplines in a project team is shared via the integrated system model repository, which acts as the “authoritative source of truth.” Within a project, there is then one authoritative repository. The one authoritative repository is a logical concept that may be implemented as a federation of distributed physical repositories. The figure schematically depicts several examples of synchronizations, milestones, and baselines (in a real project there will of course be many more). In addition, MA&S data management supporting versioning, branching, merging, and archiving needs to be

implemented for each of the threads, as well as across all organizations in an extended enterprise.

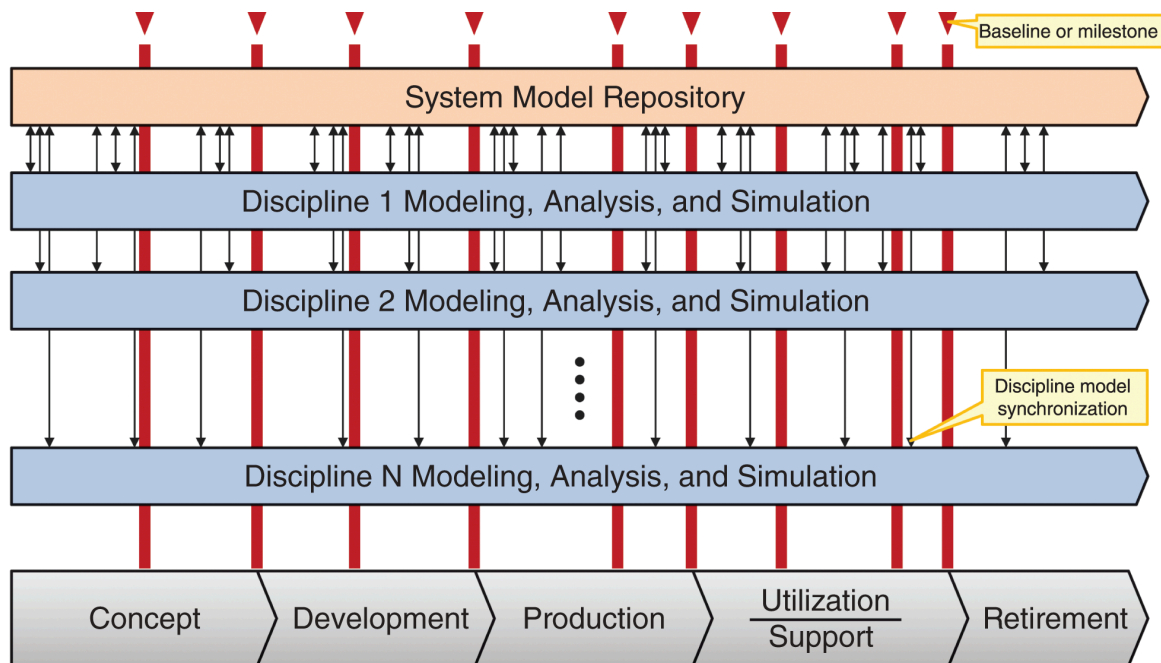


FIGURE 3.12 Multidisciplinary MA&S coordination along the life cycle. INCOSE SEH original table created by the NAFEMS/INCOSE Systems Modeling and Simulation Working Group (SMSWG). Usage per the INCOSE Notice pages. All other rights reserved.

When different organizations collaborate in an extended enterprise, the need may arise to protect intellectual property, which naturally includes the know-how captured in models. To enable such collaboration, often so-called *black box* (also known as *opaque box*) *models* are created and maintained, which hide or obfuscate intellectual property, while still providing a publicly accessible external interface for using them to perform simulations. In contrast, *white box* (also known as *transparent box*) *models* provide full visibility of their internals.

3.2.2 Prototyping

Prototyping is a technique that can significantly enhance the likelihood of providing a system that will meet the stakeholder's needs. In addition, a prototype can facilitate both the awareness and understanding of stakeholder needs and requirements. The original use of a prototype was as the first-of-a-kind product from which all others were replicated. However, prototypes are not “the first draft” of production entities. Prototypes are

intended to enhance learning and should be set aside when this purpose is achieved. Once the prototype is functioning, changes will often be made to improve performance or reduce production costs. Thus, the production entity may exhibit different behavior. Two types of prototyping are commonly used: rapid and traditional.

Rapid prototyping is an easy and one of the fastest ways to get system performance data and evaluate alternate concepts (Noorani, 2008). A rapid prototype is a particular type of physical model or simulation quickly assembled from a menu of existing physical, graphical, or mathematical elements. Examples include tools such as laser lithography or computer simulation shells. 3-D printing, or additive manufacturing, has significantly enhanced the physical elements that can be prototyped (Gebhardt and Hotte, 2016). Rapid prototypes are frequently used to investigate form and fit, human–system interface, operations, or producibility considerations. They are widely used and are particularly useful; but except in rare cases, they are not traditional “prototypes.”

Traditional prototyping is a tool that can reduce risk or uncertainty and has two primary variants. A *partial prototype* is used to verify critical elements of the system. A *full prototype* is a complete representation of the system. They must be complete and accurate in the aspects of concern. Objective and quantitative data on performance times and error rates can be obtained from higher-fidelity interactive prototypes. SE practitioners are in a much better position to evaluate modifications that will be needed to develop the system because of the existence of a traditional prototype.

3.2.3 Traceability

Traceability for products and systems is defined as “the ability to trace the history, application, or location of an object/entity/item” (ISO 9000, 2015). From an SE perspective, traceability is establishing an association or relationship between two or more objects/entities/items such as life cycle concepts, needs, requirements, architectural definition artifacts (e.g., systems, system elements), design definition artifacts, verification artifacts,

validation artifacts, information, models, and acquired or supplied systems or system elements.

Bidirectional traceability is the ability to trace an object/entity/item to another object/entity/item while automatically establishing a reverse link back to the initial object/entity/item. Thus, once a given object/entity/item has been linked to its source/destination, the source/destination is automatically linked to that object/entity/item. Bidirectional traceability is facilitated by SE tools which support the establishment of two-way links (bidirectional traceability) between objects/entities/items.

Vertical traceability is most often referred to in context of organization levels or architectural levels of the system or product under development. From a hierarchical architecture view (see [Section 1.3.5](#)), there are various system levels. The SoI level (Level n) has lower-level systems elements (Level $n+1$), some of which are further decomposed into lower-level system elements (Levels $n+2$, $n+3$, etc.) until the elements are defined to the level at which they can be made, bought, or reused. Entities at each level have objects/entities/items defined at various levels of abstraction. As the objects/entities/items are refined level-by-level, bidirectional traceability is established. Many times, these vertical traceability relations are referred to as “parent” and “child” relationships, depending upon the perspective (parent being the relationship to the higher level, child being the relationship to the lower level).

Horizontal traceability involves traceability across the elements of a given level of the architectural or system structure and across the life cycle. From a hierarchical architecture view, as relationships between objects/entities/items at the same level (i.e., Level n) are identified, bidirectional traceability is established. Many times, these horizontal traceability relations are referred to as “peer” relationships. Horizontal traceability also links objects/entities/items generated in one life cycle stage or process to data, information, and artifacts generated in other life cycle stage or process, resulting in connecting these objects/entities/items across the life cycle. For example, from a life cycle stage perspective, concept

objects/entities/items are traced to development; which are traced to production; which are traced to utilization and support; which are ultimately traced to retirement. From a life cycle process perspective, a stakeholder requirement can be traced to its system requirements; which can be traced to architecture and design artifacts; which can be traced to the realized product; which can be traced to the system verification and system validation artifacts.

Establishing traceability is a critical activity of the Technical Processes (see [Section 2.3.5](#)), especially Business or Mission Analysis; Stakeholder Needs and Requirements Definition; System Requirements Definition; System Architecture Definition; Design Definition; System Analysis; Verification; and Validation. Traceability is facilitated through the appropriate application of the Configuration Management (CM) process (see [Section 2.3.4.5](#)). The CM Identification activity enables the SE practitioner to “connect the dots” and understand the identity, location, relationships, pedigree, origin of data, materials and parts of the objects/entities/items. CM also enables the traceability of the history and location of the product after delivery. The management of products/systems, their system elements, and their configuration information requires unique identification so traceability of these items can be accurately determined. For traceability purposes the product/system identifier consists of a unique identifier which, once issued to a specific project/product/system, should never be reused.

Traceability is also a crucial component of the digital thread, enabling the connection between uniquely identified configurations of digital system models, digital twins, and physical assets (see [Section 5.4](#)). In an MBSE environment (see [Section 4.2.1](#)), the underlying system model enables a stakeholder requirement to be traced through the functional representations, to the physical product, thus enabling the identification of specific physical elements (and their specific configurations) that are impacted by a change in a given requirement. Vice versa, traceability enables the identification of the requirements that will need to be assessed when a given physical element is modified (e.g., due to a change of supplier

or manufacturing process). Digitally enabled traceability methods help ensure the stakeholders get what they asked for. Digitally enabled traceability also supports the transparency of information.

More information on traceability can be found in INCOSE GtNR (2022) and the INCOSE NRM (2022).

3.2.4 Interface Management

The purpose of Interface Management is to facilitate and manage the identification, definition, design, and management of interfaces of the system across the system life cycle. It manages interface boundaries and interactions across those boundaries, the definition and agreement for each interaction, and interface requirements for all interactions identified by the various Technical Processes. Interface Management cuts across the Agreement, Technical Management, and Technical Processes. Because of its importance, the project team should focus on Interface Management as a distinct activity across all life cycle process activities.

Given that the behavior of a system is a function of the interaction of its elements and the interaction of the SoI and external systems, it is critical for the project team to identify and define each of the interactions between all system elements that make up the integrated system as well as interactions of the integrated system with external systems and users. Failing to do so will result in costly and time-consuming rework during system integration, system verification, and system validation. Because of the criticality of interfaces, the project team must define how they will manage interfaces in their project planning (e.g., SEMP). For more complex systems, projects often develop a separate Interface Management Plan. It is often useful to have the interfaces managed using an Interface Control Working Group. Additional elaboration concerning interface identification, interface definition, interface requirements, risk assessment, and managing interfaces across the life cycle is included in the INCOSE NRM (2022).

When interface management is applied as a distinct objective and focus of the SE processes, it will help highlight underlying critical issues much earlier in the project than would otherwise be revealed that could impact the project's budget, schedule, and system performance. Identifying interface boundaries and interactions across those boundaries early in the life cycle facilitates definition of the SoI's boundaries and clarifies the dependencies the SoI has on other systems and dependencies other systems have on the SoI (see Sections 1.3.1 and 1.3.3). Identifying interface boundaries and interactions across those boundaries also helps ensure compatibility between the SoI and those external systems in which it interacts. Of particular importance is the Human Machine Interface (HMI), as ultimately it is the interaction between users, operators, and maintainers that will result in acceptance of the SoI for its intended use by its intended users (see [Section 3.1.4](#)). Failure to identify all interface boundaries and interactions across those boundaries is a significant risk to the project, especially during system integration, system verification, system validation, operations, and maintenance. Because of this, it is extremely important the project defines life cycle concepts for how it will make sure the system will work safely and securely with all the external systems and personnel with which it must interact in the intended operational environment when operated by its intended users and is protected from outside threats across those interfaces.

A key characteristic of today's increasingly complex, software-intensive systems is the number of internal interactions within systems and between a system and external systems. The increased number of interactions relates directly to the complexity of a system. It greatly increases the complexity of integration of the system elements that are part of the SoI, and integration of the realized SoI within the system it is part of. It also increases the complexity of assessing the behavior of the integrated system when operated as part of a larger system. Another key characteristic of modern software-intensive systems is the form of the interactions. In the past, when many of the systems were mostly mechanical/electrical, the interactions were more visible involving connectors, wires, pipes, mechanical parts, bolts, etc. In software-intensive systems, there can be

multiple computer modules, each with software that communicates commands, messages, and data across one or more communication busses. For example, in modern automobiles there can be more than 150 computer modules connected to each other and multiple sensors and actuators.

Interface Management Related to Life Cycle Processes

Major interface boundaries between the SoI and external systems are identified during preliminary life cycle concept definition activities within the Business or Mission Analysis process (see [Section 2.3.5.1](#)). Through the application of the Stakeholder Needs and Requirements Definition process (see [Section 2.3.5.2](#)) the life cycle concepts are further elaborated, interface boundaries between external systems are further refined to include all interface boundaries, and interactions across each of those boundaries are identified. Risks associated with each interface boundary and associated interactions are assessed as part of the Risk Management process (see [Section 2.3.4.4](#)). Using the System Requirements Definition process (see [Section 2.3.5.3](#)), the interactions are further refined and the characteristics of what is involved in each interaction are defined. Using this information, interface requirements are defined.

As the system architecture and system elements are defined, the System Architecture Definition process (see [Section 2.3.5.4](#)) concurrently with the System Requirements Definition process identify and define interface requirements for external systems, including enabling systems, which are also allocated to the applicable system elements. Internal interface boundaries and interactions across those boundaries are identified, and interface requirements defined for each of the interactions across the interface boundaries internal to the SoI (i.e., between system elements). The focus is on defining and agreeing on the characteristics of what is involved in the interactions, not on how those interactions are realized. The interface identification, definition, and requirements continue to evolve as the system requirements, architecture, design, and models evolve. These definitions are recorded in some form of interface control artifacts (e.g., Interface Control Document (ICD)) that are put under configuration control.

For each interaction across an interface boundary, the identified interaction is input to the System Requirements Definition process to define the interface requirements.

How those interactions are realized is addressed by the Design Definition process (see [Section 2.3.5.5](#)). Definitions of interactions across interface boundaries are refined to include what each system element involved in the interaction looks like at the interface boundary and the media (e.g., a data bus, a wiring harness, a physical connection, Wi-Fi, Bluetooth) involved in the interaction is determined. Additional interface boundaries and interactions may need to be identified and defined that were not addressed by the System Architecture Definition and System Requirements Definition processes. The definition of these additional interfaces often drives additional iteration between these processes to capture the interface characteristics and requirements definition. Interactions across interface boundaries are primary considerations in both horizontal and vertical integration across the life cycle as part of the Integration Process (see [Section 2.3.5.8](#)).

A major issue concerning interface definition is that when a system element is contracted out to a supplier or the SoI interacts with other supplier-developed system elements. Often the contracts are issued prior to design and thus the design definitions of what the SoI and system elements look like at the interface boundary and the media involved in the interaction have not yet been defined. In addition, it is common for the suppliers to have little insight into the workings of other supplier-developed system elements with which they interact and how changes to those system elements could affect the interactions and performance of their system element (or changes to their system element could affect other system elements). In these cases, it is important that the acquirer clearly addresses how each supplier will support, participate in, and comply with the interface management activities during interface definition, design, system integration, system verification, and system validation in the agreements via the Agreement Processes (see [Section 2.3.2](#)).

Using the System Analysis process (see [Section 2.3.5.6](#)), the level and type of analysis needed to understand the trade space with respect to the interface requirements and definition is determined and performed. This can include mathematical analysis, modeling, simulation, experimentation, and other techniques. The analysis results are input to trade-offs made through the Decision Management process (see [Section 2.3.4.3](#)).

The Implementation process (see [Section 2.3.5.7](#)) is used to develop the system element interfaces and record evidence of meeting the interface requirements for an implemented system element.

The Integration process (see [Section 2.3.5.8](#)) considers the integration of the system and system element interfaces in the integration planning and integrates the implemented system elements together at the interface boundaries. Each system element is verified to have met their interface requirements using the Verification process (see [Section 2.3.5.9](#)). The system interfaces are validated using the Validation process (see [Section 2.3.5.11](#)) against the stakeholder needs and stakeholder requirements concerning interactions with systems or users external to the SoI in the operational environment. The Transition process (see [Section 2.3.5.10](#)) checks the installation and operational state of the interfaces in the operational environment.

Key activities that are part of interface management include facilitating cooperation and agreements with other stakeholders, defining roles and responsibilities, enabling open communication concerning issues, establishing timing for providing interface information, problem resolution, and agreeing on the interaction characteristics across interface boundaries early in the project. These functions are done through the Project Planning process (see [Section 2.3.4.1](#)). An important interface analysis activity is assessing and managing risks as part of the Risk Management process (see [Section 2.3.4.4](#)), avoiding potential impacts especially during system integration, system verification, system validation, operation, and maintenance. Other processes may also contribute to the management of the interfaces.

After establishing baselines for interface requirements, interface definitions, architecture, and design, the Configuration Management process (see [Section 2.3.4.5](#)) provides the ongoing management and control of the interface requirements and definitions, as well as any associated artifacts.

Recording Definitions of Interactions across Interface Boundaries

In a document-centric practice of SE, definitions concerning interface boundaries, the interaction across those boundaries, and the media involved are commonly recorded in some type of interface definition artifact (e.g., Interface Control Document (ICD), Data Dictionary (DD), Interface Definition Document (IDD), Interface Agreement Document (IAD)) or within the project's integrated dataset from which the associated report may be generated. In a data-centric practice of SE, these are often captured in databases and models. A data-centric practice enables effective impact and change analysis, as well as helping ensure consistency of interface requirements and definitions across the architecture.

Interface Analysis

In conjunction with the other Technical Processes, the System Analysis process (see [Section 2.3.5.6](#)) applies various analysis methods to identify interface boundaries, and interactions across those boundaries, to better understand how the SoI interacts with the other systems that make up the system of which it is a part and to help ensure there are no missing interface boundaries, definitions, and interface requirements. Some common diagrams, methods, and tools used for analysis include functional flow block diagrams (FFBD), data flow diagrams (DFD), context diagrams, boundary diagrams, external interface diagrams, input, process, output (IPO) diagrams, N^2 diagrams, and internal interface diagrams, Failure Modes and Effects Analysis (FMEA), System-Theoretic Process Analysis (STPA), language-based models (e.g., SysML diagrams), and simulations.

A critical part of interface analysis includes an assessment of each interaction across an interface boundary in terms of maturity, stability, documentation, threats, and risks. The SoI is particularly vulnerable when interfacing with external systems over which they may have little or no control. Because of this, the SoI is vulnerable to undesirable effects at and across the interface boundaries. Therefore, identifying and managing risks associated with interface boundaries and interactions across those boundaries is key to exposing potential risks to the project across applicable life cycle stages. Many of the major issues discovered during system integration, system verification, and system validation involve interfaces.

An example analysis tool is the N^2 diagram shown in [Figure 3.13](#), which enables a systematic approach to identifying interface boundaries and interactions across those boundaries. N^2 diagrams enable the SE practitioner to assess and identify interface boundaries and interactions across those boundaries in a structured, bidirectional, fixed framework. The N^2 diagrams can be used at several levels of abstraction of the SoI: a functional view and a physical view.

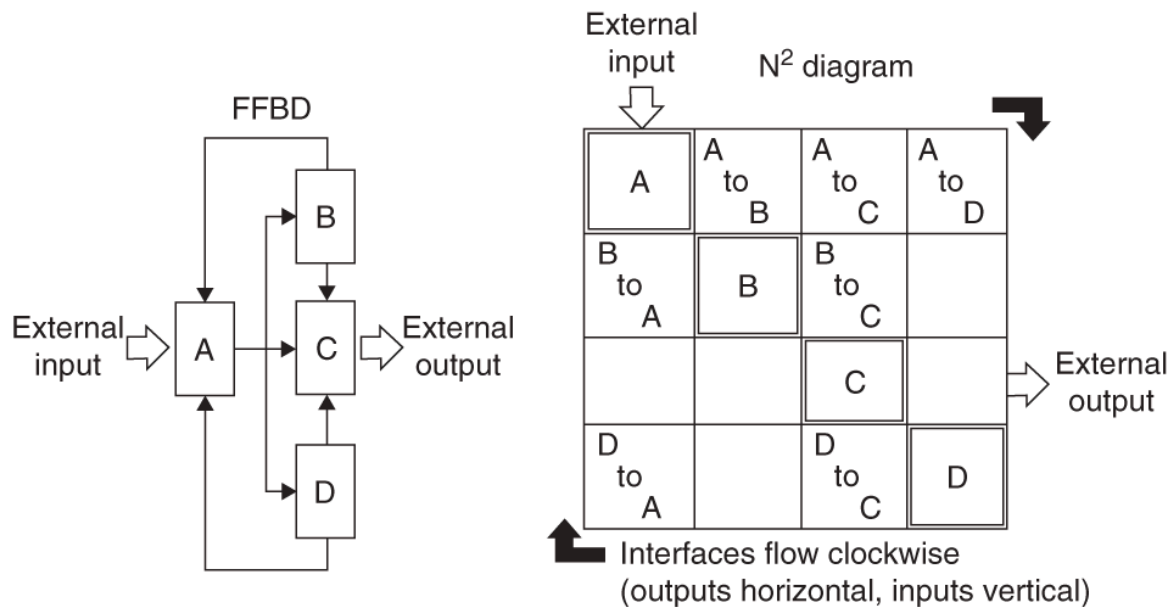


FIGURE 3.13 Sample N-squared diagram. INCOSE SEH original figure created by Krueger and Forsberg. Usage per the INCOSE Notices page. All other rights reserved.

An N^2 diagram is created using an $N \times N$ matrix. The system elements (functional or physical) are placed in squares forming a diagonal from upper left to lower right. The rest of the squares in the matrix represent potential interactions (interfaces) between the elements. In an N^2 diagram, interactions between elements flow in a clockwise direction. For example, the entity being passed from element A to element B, can be defined in the appropriate off-diagonal square. A blank square indicates there is no interaction between the respective elements. Sometimes, characteristics of the entity passing between elements may be included in the *off-diagonal* square where the interacting entity is identified. When all elements have been compared to all other elements, then the matrix is complete. If lower-level elements are identified in the process with corresponding lower-level interactions, then they can be successively described in expanded lower-level N^2 diagrams. The Design Structure Matrix (DSM) is very similar in appearance and usage to the N^2 diagram, but a different input and output convention is typically used (inputs on the horizontal rows and outputs on the vertical columns) resulting in interactions between elements flowing in a counterclockwise direction (Eppinger and Browning, 2012). Figure D-1 illustrates an N^2 diagram for the interactions amongst the system life cycle processes.

One of the main functions of the N^2 diagram, besides the identification of interactions, is to pinpoint areas where conflicts may arise between elements so that systems integration later in the development cycle can proceed efficiently (Becker, et al., 2000) (DSMC, 1983) (Lano, 1977). Alternatively, or in addition, functional and physical diagrams can be used with N^2 diagrams to characterize the flow of information among system elements and between system elements and the external systems. As the system architecture is decomposed to lower levels, it is important to ensure the interface interaction definitions keep pace and that interactions are defined so that decompositions of lower levels are considered.

Coupling matrices (a type of N^2 diagram, shown in [Figure 3.14](#)) are a basic method to define the aggregates and the order of integration (Grady, 1994). They can be used during System Architecture Definition (see [Section 2.3.5.4](#)), with the goal of keeping the interfaces as simple as possible. Simplicity of interfaces can be a distinguishing characteristic and a selection criterion between alternate architectural candidates. The coupling matrices are also useful for optimizing the aggregate definition and the verification of interfaces during Integration (see [Section 2.3.5.8](#)). Integration can be optimized by reorganizing the coupling matrix in order to group the system elements into aggregates and minimize the number of interfaces to be verified between aggregates. When verifying the interactions between aggregates, the coupling matrix can be an aid for fault detection.

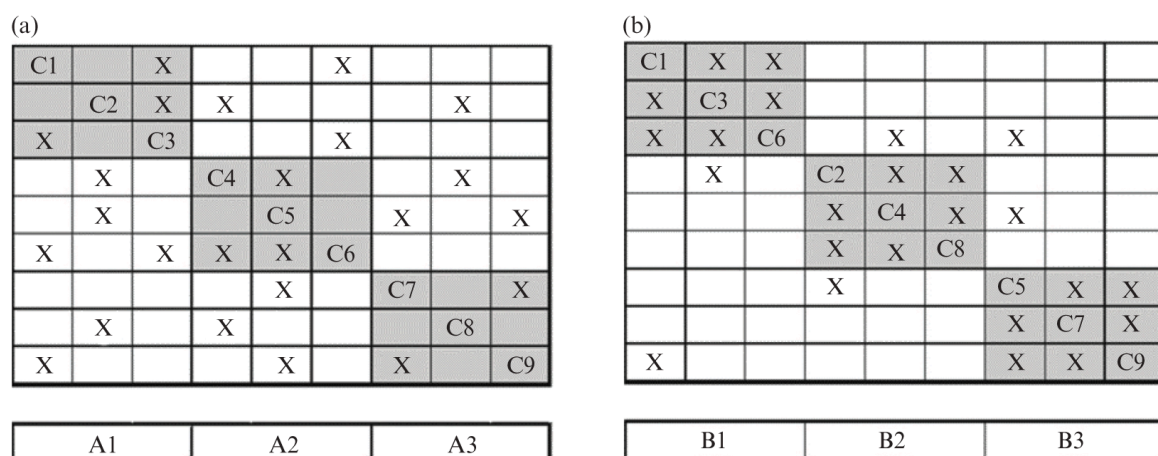


FIGURE 3.14 Sample coupling matrix showing: (a) Initial arrangement of aggregates; (b) final arrangement after reorganization. INCOSE SEH original figure created by Faisandier. Usage per the INCOSE Notices page. All other rights reserved.

3.2.5 Architecture Frameworks

An *architecture description framework* is defined in ISO/IEC/IEEE 42010 (2022) as:

A set of “conventions, principles and practices for the description of architectures established within a specific domain of application or community of stakeholders.”

The term “description” is used in the definition to avoid confusion between architecture description frameworks and other frameworks (e.g., enterprise architecture framework, architecture evaluation framework). Other definitions for *architecture framework* (AF) can be found in the technical literature, for example The Open Group Architecture Framework (OMG TOGAF, 2023) defines AF as:

A foundational structure, or set of structures, which can be used for developing a broad range of different architectures.

Architecture frameworks are used in various domains to help ensure harmonization, consistency, and re-use. When a commonly agreed upon architecture framework is adhered to by all project teams involved, better aligned project artifacts typically result. This benefit will be particularly evident in distributed teams and within enterprises when architecture descriptions and architectural artefacts are reused across projects.

Most architecture frameworks are organized to provide one or more *viewpoints* to cover the target domains and their typical stakeholders’ concerns (e.g., NATO AF (NAF), Unified AF (UAF), and Department of Defense AF (DoDAF)). Some frameworks also provide one or more of the following:

- A method for describing systems in terms of a set of architecture building blocks, and for showing how the building blocks fit together.
- A set of tools and a common vocabulary.

- Multiple dimensions with coordinates for relating particular groups of concerns or solutions along the dimensional aspects.

Figure 3.15 provides an overview of the Unified Architecture Method (UAM), which provides dimensions for perspectives and aspects.

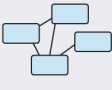
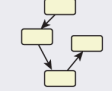
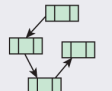
	Aspect			
Perspective	Data	Activity	Location	People
Business	 Business Entity Model	 Business Process Model	 Business Locations Model	 Business Roles Model
Logical	 Logical Entity Model	 Logical Process Model	 Logical Locations Model	 Logical Roles Model
Technical	 Technical Entity Model	 Technical Process Model	 Technical Locations Model	 Technical Roles Model

FIGURE 3.15 Unified Architecture Method. From UAM (2022). Used with permission. All other rights reserved.

Others advocate that architecture frameworks should include a list of recommended standards, libraries of patterns, and compliant products that can be used to accelerate architecting. Finally, it is useful for architecture frameworks, or more broadly, architecting environments, to define activities and resources for architecture governance, in addition to the governance of skills and competencies in place, with regard to enterprise objectives.

Framework Support to Architecture Activities

Architecture activities are described in the System Architecture Definition process (see [Section 2.3.5.4](#)). This section explains how some of the major architecture frameworks can be used to perform the key architecture activities.

Architecture Enablement Frameworks like Pragmatic Enterprise AF (PEAF) (Pragmatic 365, 2023) and Generalized Enterprise Reference Architecture